```
 1
 2  // COS30008, Problem Set 4, Problem 1, 2022
 3
 4  #pragma once
 5
 6  #include <algorithm>
 7  #include <cstddef>
 8  #include <stdexcept>
 9  #include <utility>
10
11  template <typename T> struct BinaryTreeNode {
12      using BNode = BinaryTreeNode<T>;
13      using BTreeNode = BNode *;
14
15      T key;
16      BTreeNode left;
17      BTreeNode right;
18
19      static BNode NIL;
20
21      const T &findMax() const {
22          if (empty()) {
23              throw std::domain_error("Empty tree encountered.");
24          }
25
26          return right->empty() ? key : right->findMax();
27      }
28
29      const T &findMin() const {
30          if (empty()) {
31              throw std::domain_error("Empty tree encountered.");
32          }
33
34          return left->empty() ? key : left->findMin();
35      }
36
37      bool remove(const T &aKey, BTreeNode aParent) {
38          BTreeNode x = this;
39          BTreeNode y = aParent;
40
41          while (!x->empty()) {
42              if (aKey == x->key) {
43                  break;
44              }
45
46              y = x; // new parent
47
48              x = aKey < x->key ? x->left : x->right;
49          }
```

```
50
51          if (x->empty()) {
52              return false; // delete failed
53          }
54
55          if (!x->left->empty()) {
56              const T &lKey = x->left->findMax(); // find max to left
57              x->key = lKey;
58              x->left->remove(lKey, x);
59          } else {
60              if (!x->right->empty()) {
61                  const T &lKey = x->right->findMin(); // find min to right
62                  x->key = lKey;
63                  x->right->remove(lKey, x);
64              } else {
65                  if (y != &NIL) // y can be NIL
66                  {
67                      if (y->left == x) {
68                          y->left = &NIL;
69                      } else {
70                          y->right = &NIL;
71                      }
72                  }
73
74                  delete x; // free deleted node
75              }
76          }
77
78          return true;
79      }
80
81      // PS4 starts here
82
83      // is nil supposed to be whatever it is before they invented nullptr?;
84
85      // I love how the type aliasing makes thing MUCH MORE CONFUSING
86
87      BinaryTreeNode() {
88          this->left = &NIL;
89          this->right = &NIL;
90      }
91      BinaryTreeNode(const T &aKey) {
92          // copy constructor?
93          this->key = aKey;
94          this->left = &NIL;
95          this->right = &NIL;
96      };
97      BinaryTreeNode(T &&aKey) {
98          this->key = std::move(aKey);
```

```
 99          this->left = &NIL;
100          this->right = &NIL;
101      };
102
103      ~BinaryTreeNode() {
104          if (!this->left->empty()) {
105              delete left;
106          }
107          if (!this->right->empty()) {
108              delete right;
109          }
110      };
111
112      bool empty() const { return this == &NIL; };
113      bool leaf() const { return this->left == &NIL && this->right ==
           &NIL; };
114      size_t height() const {
115          if (this->empty()) {
116              throw std::domain_error("Empty tree encountered");
117          } else if (this->leaf()) {
118              return 0;
119          } else {
120              size_t t = 0;
121              if (!this->left->empty()) {
122                  t = std::max(t, this->left->height() + 1);
123              }
124              if (!this->right->empty()) {
125                  t = std::max(t, this->right->height() + 1);
126              }
127              return t;
128          }
129      };
130
131      bool insert(const T &aKey) {
132          // auto node = new BinaryTreeNode(aKey);
133          if (this->empty()) {
134              return false;
135          } else {
136              auto node = this;
137              while (!node->empty()) {
138                  if (aKey == node->key) {
139                      return false;
140                  } else if (aKey < node->key) {
141                      if (node->left->empty()) {
142                          node->left = new BinaryTreeNode<T>(aKey);
143                          return true;
144                      } else {
145                          node = node->left;
146                      }
```

```
147                 } else if (aKey > node->key) {
148                     if (node->right->empty()) {
149                         node->right = new BinaryTreeNode<T>(aKey);
150                         return true;
151                     } else {
152                         node = node->right;
153                     }
154                 }
155             }
156             return false;
157         }
158     };
159 };
160 
161 template <typename T> BinaryTreeNode<T> BinaryTreeNode<T>::NIL;
162 
```